

Cricket Batting Stance Analyzer — Complete Project Capsule

Created: May 2026 | Author: Suhas | Use this file in: ChatGPT, Gemini, Claude, Copilot

HOW TO USE THIS FILE

Paste the entire contents of this file into any AI chat (ChatGPT, Gemini, etc.) and say:

┆ "I am building this project. Here is my full project context. Help me with [what you need]."

The AI will have full context of everything — what you've built, what you're building, your stack, your dataset strategy, your research papers, and your Week 1 plan.

1. WHAT THE PROJECT IS

One-line description

An AI-powered cricket batting stance analyzer that takes a 30-second phone video and outputs a 4-factor coaching report — no hardware, no subscription, no coach needed.

The core idea

Upload a batting video → system extracts 7 key frames (stance → trigger → backlift → contact → follow-through) → MediaPipe detects 33 body joint keypoints → AI analyzes technique → outputs 4 coaching factors.

The 4-factor output (what makes it unique)

1. **WEAKNESS** — A technical flaw + the exact delivery type that exploits it (e.g. "Head falling to off side → vulnerable to inswing on off stump line")
2. **STRENGTH** — A positive element + why it generates power/control (e.g. "High bat speed at contact → generates power even on off-pace deliveries")
3. **THING TO CHANGE** — A specific physical instruction + drill (e.g. "Redirect backlift towards fine leg — drill: tap a stump placed at fine leg on every backlift rep")
4. **BONUS INSIGHT** — Additional observations (e.g. "Trigger movement towards off stump reduces LBW protection on leg stump line")

Why it's unique vs market

Competitor	What they do	What they miss
StanceBeam	Bat sensor — measures bat speed, angle	Needs ₹5,299 hardware. No coaching language.
CricVision	Skeleton video analysis for academies	B2B only. No delivery vulnerability prediction.
Fulltrack AI	Ball tracking + shot cues (\$9.99/mo)	No stance coaching. No weakness→delivery mapping.
Research papers	Shot classification, phase detection	None output coaching language or delivery mapping.

Your unique features no one has:

- Delivery vulnerability prediction (flaw → specific bowling threat)
- 4-factor coaching output in plain English
- 7-frame coaching window (stance to contact)
- Free, phone-only, grassroots accessible
- LLM-bootstrapped labelling pipeline (novel data approach)

2. TECH STACK

Frontend

- React + Vite
- Tailwind CSS
- ffmpeg.wasm (in-browser video frame extraction — no backend needed for this step)

Backend

- FastAPI (Python)
- Uvicorn ASGI server
- Hosted on Azure B1s VM (free tier, 12 months)

AI — Phase 1 (current)

- Gemini Vision API (gemini-2.0-flash) — free tier, 1500 calls/day
- MediaPipe Pose — 33 body joint keypoints per frame

AI — Phase 2 (building towards)

- Option A: Dense keypoint classifier (Keras) — train first, needs 150+ samples
- Option B: MobileNetV2 transfer learning (Keras) — main model, needs 300+ samples
- Option C: CNN + LSTM sequence model — future, needs 500+ sequences

Cloud — Azure (free tier)

- **Azure Blob Storage** — raw_videos/, frames/, dataset/ containers
- **Azure B1s VM** — runs pipeline.py + FastAPI backend
- **Azure Static Web Apps** — hosts React frontend (free)
- **GitHub Actions** — CI/CD pipeline for automated deployments

Dataset pipeline

- yt-dlp — auto-downloads YouTube batting videos on Azure VM
- ffmpeg — extracts 7 frames per video
- MediaPipe — extracts 33 joint keypoints per frame → keypoints.csv
- Gemini Vision API — auto-labels frames with 4-factor coaching JSON → labels.csv
- merge.py — joins keypoints + labels → dataset.csv (training file)

3. ARCHITECTURE — HOW IT ALL CONNECTS

```
User phone video
    ↓
React frontend (ffmpeg.wasm extracts 7 frames in-browser)
    ↓
FastAPI backend on Azure VM
    ↓
MediaPipe (33 keypoints per frame) + Gemini Vision (reads frames visually)
    ↓
4-factor JSON response
    ↓
React frontend displays: Weakness / Strength / Change / Bonus + scores
```



Azure Blob Storage (saves frames + labels silently as training data)

4. THE 7-FRAME SEQUENCE

Frame	Phase	What is detected
Frame 1	Initial stance	Foot position, guard, head alignment, bat at rest, shoulder angle
Frame 2	Trigger movement	Front foot direction, head drift, early weight shift
Frame 3	Backlift start	Bat lift angle, top hand, elbow height
Frame 4	Full backlift	Peak backlift height, bat face direction, wrist position
Frame 5	Downswing	Bat path, weight transfer, head over ball
Frame 6	Ball contact	Bat face at impact, head position, knee bend
Frame 7	Follow-through	Bat swing direction, hip rotation, balance

5. DATASET STRATEGY

Target: 500 labelled sessions

- 200 sessions → beginner / gully level
- 150 sessions → club / district level
- 80 sessions → state / semi-pro
- 70 sessions → professional (YouTube slow-mo footage)

Angle split

- 250 sessions → side-on (most valuable for backlift, weight transfer)
- 175 sessions → front view (easier to collect, good for head/shoulder/stance width)
- 75 sessions → 45° diagonal

Current status

- ~20 videos collected (friends' batting, mostly front view)

- Azure Blob Storage being set up
- auto_download_azure.py will collect 150+ beginner YouTube videos overnight

Data sources

1. Own recordings of friends batting (current: ~20)
2. YouTube: "gully cricket batting India", "tape ball cricket batting", "cricket net practice session", "batting tutorial for beginners Hindi"
3. CricketVision dataset (WACV 2025) — email Tevin Moodley at University of Johannesburg
4. Augmentation: horizontal flip doubles dataset (right-handed → left-handed) instantly

Label format (per session JSON saved to Azure Blob)

```
json

{
  "session": "arjun_front_beginner_01",
  "weakness": ["Head falling to off side - creates inswing vulnerability"],
  "strength": ["Good knee bend at stance - allows fast foot movement"],
  "change": ["Redirect backlift to fine leg direction - tap stump drill"],
  "bonus": ["Trigger moves toward off stump - LBW risk on leg stump"],
  "scores": {"balance": 62, "power": 78, "technique": 55, "defence": 70}
}
```

File naming convention

```
arjun_front_beginner_01.mp4 → [name]_[angle]_[skill]_[number].mp4
angle options: front / side / diagonal
skill options: beginner / club / state / pro
```

6. AZURE SETUP (what's been set up / needs to be set up)

Commands to set up from scratch

```
powershell
```

```
# Install Azure CLI
winget install Microsoft.AzureCLI

# Login
az login

# Create resource group (South India = closest to Bengaluru)
az group create --name cricket-rg --location southindia

# Create storage account (change "suhascrkt" to unique name)
az storage account create --name suhascrkt --resource-group cricket-rg --location southindia

# Get connection string – SAVE THIS OUTPUT
az storage account show-connection-string --name suhascrkt --resource-group cricket-rg

# Create containers
az storage container create --name raw-videos --account-name suhascrkt
az storage container create --name frames --account-name suhascrkt
az storage container create --name dataset --account-name suhascrkt

# Create free B1s VM
az vm create --resource-group cricket-rg --name cricket-vm --image Ubuntu2204 --size Standard_B1s
```

Stop VM when done (avoids burning free hours)

```
bash

az vm deallocate --resource-group cricket-rg --name cricket-vm
```

Start VM when needed

```
bash

az vm start --resource-group cricket-rg --name cricket-vm
```

7. SCRIPTS BUILT SO FAR

extract_frames.py

Extracts 7 evenly spaced frames from each .mp4 in raw_videos/. Saves as frame_01.jpg to frame_07.jpg in frames/{session_name}/

extract_keypoints.py

Runs MediaPipe Pose on every frame. Saves 33 joint x,y coordinates per frame to keypoints.csv. 1 row per frame.

auto_label_gemini.py

Sends all 7 frames per session to Gemini Vision API. Gets back 4-factor JSON coaching labels. Saves to labels.csv. Auto-resumes if interrupted.

merge.py

Joins keypoints.csv + labels.csv on session name → dataset.csv (your training file).

auto_download_azure.py

Runs on Azure VM. Downloads batting videos from YouTube using yt-dlp, uploads to Azure Blob, deletes from VM immediately. Runs overnight.

pipeline.py (Azure version)

Full cloud pipeline: reads videos from Azure Blob → extracts frames → runs MediaPipe → Gemini labels → saves everything back to Azure Blob. Zero laptop storage used.

8. RESEARCH PAPERS (cite these in your project)

Paper	Venue	Key finding	Your gap over it
CricTAL — Temporal Activity Localisation for Cricket Batting	ICCV Workshops 2025	TCN achieves 90.4% accuracy detecting phase boundaries in batting	Only finds WHEN phases happen. No technique quality assessment. No coaching output.
Enhancing Cricket Performance Analysis with HPE and ML	Sensors 2023	99.77% accuracy predicting 8 stroke types using MediaPipe + Random Forest	Classifies shot TYPE not technique quality. No weakness/strength output.
Hierarchical NeuroSymbolic AQA	AAAI 2024	Interpretable coaching reports with 1-degree angular precision for Olympic diving	For diving only. No cricket. No delivery vulnerability prediction exists anywhere.
Action Quality Assessment via	arxiv Jan 2025	Multi-scale visual-skeleton encoder for	Never applied to cricket. Your 7-frame window is your version of

Paper	Venue	Key finding	Your gap over it
Hierarchical Pose-guided Contrastive Regression		fine-grained AQA	their procedure segmentation.
Learning To Score Olympic Events	arxiv	LSTM for improvement feedback on diving scores	Your NLP coaching output is more interpretable and actionable.
Backlift Recognition with Xception	Scientific Reports 2022	98.2% accuracy detecting lateral vs straight backlift	Single feature only. No downstream impact prediction.

9. MODELS — 4 OPTIONS (in build order)

Option A — Dense Keypoint Classifier (build FIRST, Week 5)

- **Input:** 66 numbers per frame ($33 \text{ joints} \times x,y$) $\times$ 7 frames = 462 features
- **Architecture:** Dense(256, relu) $\rightarrow$ Dropout(0.3) $\rightarrow$ Dense(128, relu) $\rightarrow$ Dense(4, sigmoid)
- **Training data needed:** 150+ sessions
- **Training time:** Under 5 minutes on CPU
- **Accuracy expectation:** 70-80% agreement with Gemini labels

Option B — MobileNetV2 Transfer Learning (build second, Week 6-7)

- **Input:** Raw 224 $\times$ 224 JPEG frames
- **Architecture:** MobileNetV2 (frozen) $\rightarrow$ GlobalAveragePooling2D $\rightarrow$ Dense(256, relu) $\rightarrow$ Dense(4, sigmoid)
- **Training data needed:** 300+ sessions
- **Training time:** 20-40 min on Colab GPU
- **Accuracy expectation:** 80-88%
- **Udemy connection:** Directly uses transfer learning lessons — freeze base, add custom head, fine-tune

Option C — CNN + LSTM Sequence Model (future)

- **Input:** 7 frames as temporal sequence
- **Architecture:** TimeDistributed(MobileNetV2) $\rightarrow$ LSTM(128) $\rightarrow$ Dense(4, sigmoid)
- **Training data needed:** 500+ full sequences
- **Udemy connection:** Combines CNN transfer learning + LSTM module

Option D — Pose Graph Neural Network (research level)

- **Input:** MediaPipe keypoints as graph (33 nodes, bones as edges)
 - **Architecture:** Graph Convolutional Network → GlobalPooling → Dense output
-

10. UNIQUE FEATURES THAT MAKE IT EYE-CATCHING

1. **Delivery vulnerability heatmap** — SVG wagon-wheel showing which deliveries are dangerous vs safe based on detected stance flaws. Nobody has this.
 2. **Skeleton overlay visualisation** — MediaPipe's 33 keypoints drawn on frame, bad joints in red, good in green. Shows the model isn't a black box.
 3. **Azure ML model versioning** — Dense v1 → MobileNetV2 v2 registered in Azure ML workspace. How real production ML systems work.
 4. **Progress tracker** — Compare this week's session vs last month. Show improvement over time in each dimension.
 5. **CI/CD pipeline** — GitHub Actions auto-deploys to Azure on every push. One YAML file. Massive resume talking point.
 6. **Model accuracy dashboard** — React page showing Dense vs MobileNetV2 vs Gemini baseline accuracy. Recharts graphs.
-

11. COMMERCIAL VALUE

- **Freemium:** Free 3 analyses/month → ₹99-199/month unlimited. 10,000 users = ₹9.9L/month
 - **B2B academies:** ₹2,000-5,000/month per academy. 100 academies = ₹3L/month
 - **State board licensing:** BCCI state associations for talent identification. ₹10-50L annually
 - **API licensing:** POST /predict endpoint → sports tech companies pay per-call
-

12. RESUME BULLET POINTS

Cricket Batting Stance Analyzer | Python · TensorFlow · MobileNetV2 · Azure · React

- Built and trained a MobileNetV2 transfer learning model on a custom dataset of 500+ cricket batting

sessions, achieving [X]% accuracy in predicting 4 coaching factors: weakness, strength, improvement drill, and delivery vulnerability

- Architected end-to-end MLOps pipeline on Azure: Blob Storage for dataset (500+ sessions), B1s VM for model serving, Static Web Apps for frontend, GitHub Actions CI/CD for automated deployment
- Designed automated dataset collection pipeline using ffmpeg.wasm (frame extraction), MediaPipe Pose (33-keypoint extraction), Gemini Vision API (auto-labelling) – bootstrapping 500+ labelled training samples with zero manual annotation
- Developed delivery vulnerability prediction module mapping batting technique flaws to specific bowling threats – feature not present in any published research paper or commercial product
- Deployed full-stack web app (React + FastAPI) with real-time 7-frame extraction, MediaPipe skeleton overlay visualisation, and AI coaching feedback targeting 300M+ grassroots cricketers

13. WEEK 1 PLAN — STARTING NOW

Goal by end of Week 1: Working React app with all 4 stance form steps + Claude/Gemini API wired + 4-factor results displaying on screen. Azure setup complete. Videos uploading to cloud.

Day 1 — Environment setup

- Sign up Azure (azure.microsoft.com/en-us/free/students with college email)
- Run Azure setup commands (resource group + blob storage + containers + VM)
- Install Node.js v20 LTS from nodejs.org
- Install Python 3.11 from python.org (tick "Add to PATH")
- Install VS Code + extensions: ESLint, Prettier, Python, Tailwind IntelliSense
- Get Gemini API key from aistudio.google.com (free)
- Create GitHub repo: cricket-stance-analyzer (public)
- Upload existing 20 videos to Azure Blob via Azure Storage Explorer app

Day 2 — Scaffold React project

- `npm create vite@latest frontend` → choose React + JavaScript
- `cd frontend && npm install && npm run dev`
- Install Tailwind CSS: `npm install -D tailwindcss postcss autoprefixer && npx tailwindcss init -p`
- Create folder structure: `src/components/form/`, `src/components/results/`, `src/api/`, `src/prompts/`
- Delete boilerplate → write `<h1>Cricket Stance Analyzer</h1>` → confirm on screen
- Create .env file: `VITE_GEMINI_KEY=your-key` → add .env to .gitignore immediately

Day 3 — Build form Steps 1 & 2

- ProfileStep.jsx: batting hand, playing level, preferred style (3 select dropdowns)
- StanceStep.jsx: foot position, stance width, guard, head position, knee bend, weight, shoulders (option button grids)
- App.jsx: single stanceData state object, pass setStanceData as props
- Step navigation: currentStep state (0-3), Next/Back buttons
- Test: add `<pre>{JSON.stringify(stanceData, null, 2)}</pre>` → confirm state updates on click

Day 4 — Build form Steps 3 & 4

- GripStep.jsx: grip type, backlift direction, bat at rest
- Multi-select trouble areas (array toggle for multiple selections)
- AnalyzeStep.jsx: summary of all selections + "Analyze my stance" button
- Test full flow: all 4 steps, navigate back, confirm data persists

Day 5 — Wire Gemini API

- Create `src/api/geminiClient.js` with `analyzeStance(stanceData)` function
- Create `src/prompts/stancePrompt.js` — `buildPrompt(stanceData)` returns 4-factor JSON request
- Prompt must ask Gemini to return: `{"weakness":[], "strength":[], "change":[], "bonus":[], "scores":{}}`
- Use `import.meta.env.VITE_GEMINI_KEY` — never hardcode the key
- Test: call `analyzeStance()` on button click → check browser console for real 4-factor JSON

Day 6 — Build results display

- ResultsPanel.jsx: accepts Claude/Gemini JSON as prop
- 4 coloured cards: red (Weakness), green (Strength), amber (Change), purple (Bonus)

- Score cards: balance, power, technique, defence with colour bars (green ≥ 75 , amber ≥ 50 , red below)
- Wire: Analyze button $\rightarrow$ setLoading(true) $\rightarrow$ analyzeStance() $\rightarrow$ setResult(data) $\rightarrow$ show ResultsPanel
- Add loading spinner while waiting for Gemini (5-15 seconds)

Day 7 — Polish + deploy to Azure + push to GitHub

- Error handling: try/catch, friendly error message on API failure
 - Test on mobile: Chrome DevTools 375px width $\rightarrow$ fix overflow
 - Deploy frontend to Azure Static Web Apps (free): `az staticwebapp create`
 - First commit and push to GitHub: `git add . && git commit -m "week 1 complete" && git push`
 - Share live URL — Week 1 done ✓
-

14. KEY COMMANDS REFERENCE

Azure

```
bash

# SSH into VM
ssh azureuser@YOUR_VM_IP

# Stop VM (stops billing)
az vm deallocate --resource-group cricket-rg --name cricket-vm

# Start VM
az vm start --resource-group cricket-rg --name cricket-vm

# Upload video to blob
az storage blob upload --container-name raw-videos --account-name suhascrkt --file "

# Check blob contents
az storage blob list --container-name dataset --account-name suhascrkt --output tabl
```

Python pipeline (run on Azure VM via SSH)

```
bash
```

```
# Install packages
pip3 install mediapipe opencv-python-headless azure-storage-blob google-generativeai

# Run pipeline in background
nohup python3 pipeline.py > pipeline.log 2>&1 &

# Check progress
tail -f pipeline.log

# Run auto-download overnight
nohup python3 auto_download_azure.py > download.log 2>&1 &
```

Windows PowerShell — set Gemini key

```
powershell

$env:GEMINI_API_KEY="AIzaSy...your-key"
```

Git

```
bash

git add .
git commit -m "your message"
git push origin main
```

15. PEOPLE / RESOURCES TO CONTACT

- **Tevin Moodley** — University of Johannesburg — email for CricketVision dataset (8,540 annotated cricket clips, WACV 2025). Say you're a student building a batting stance analyzer.
 - **CricShot10 dataset** — email aniksen.cuet09@gmail.com — 10 cricket shot categories from YouTube
 - **Roboflow** — universe.roboflow.com/yolo-qyrvs/cricket-batsman-stance — 156 images, instant download
-

16. IMPORTANT NOTES

1. **Don't use raw API key in frontend** — always use `import.meta.env.VITE_KEY` and keep `.env` in `.gitignore`
2. **Azure VM** — always run `az vm deallocate` when done, not just disconnect from SSH
3. **Gemini free tier** — 15 requests/minute, 1500/day. Script sleeps 4 seconds between calls automatically
4. **MobileNet can't be trained without data** — you need 300+ sessions first. Use Gemini API for production while collecting data.
5. **Front view videos are fine** — MediaPipe still detects head, shoulder alignment, stance width, knee bend from front view (4 of 6 key detections). Side-on is better but don't wait for it.
6. **Horizontal flip augmentation** — 250 real sessions → run `augment.py` → 500 sessions instantly. Right-handed flipped = left-handed. Free data.
7. **Dataset naming** — always use `[name]_[angle]_[skill]_[number].mp4` format so pipeline tags angle and skill level automatically

End of capsule — version 1.0 — May 2026 Project: Cricket Batting Stance Analyzer / Builder: Suhas / Stack: React + FastAPI + MobileNetV2 + Azure + Gemini